

Tunable Client-Centric Consistency in a MongoDB Replica Set

DSA5208 Project 1 - Final Report

Zhao Yikun | Zhou Jiahao | Chen Zheke

23 September 2026

Abstract

This project investigates how MongoDB consistency controls affect four client-centric guarantees: read-your-writes (RYW), monotonic reads (MR), monotonic writes (MW), and writes-follow-reads (WFR). We deployed MongoDB 8.0.12 as a three-node replica set in Docker and compared four configurations that vary read concern, write concern, read preference, and use of an explicit causally consistent session. Experiments covered normal operation, one- and two-node loss, Primary failover, and replication-network partition. Each trial used versioned documents and explicit dependency fields, while timeout, rollback, and recovery were reported separately from consistency violations.

The strongest contrast was RYW under a stopped Secondary: C1, using local reads, $w=1$, secondaryPreferred and no explicit causal session, returned a stale version in 70 of 100 evaluable formal trials; C4, using majority concerns and one causal session, returned no stale version in 100. MR produced no regressions in 400 formal S2/S4 trials, while a targeted partition diagnostic produced five regressions in five C1 trials and five C4 timeouts. The fixed WFR fault matrix completed 600 formal attempts: 596 were evaluable, no confirmed or candidate violation was observed, four were inconclusive, 1,198 of 1,200 workload operations succeeded, and every trial recovered. MW experiments found no confirmed client write-order violation, but loss of a majority caused timeouts and unavailable outcomes. The observations generally agree with the predictions, while also showing that durability, causal ordering, availability, and recovery are different properties.

1. Project scope and research questions

The project asks how application-visible consistency changes when a replicated database varies acknowledgement, visibility, routing, and causal-session settings. The four investigated guarantees are:

- **RYW:** after a client receives acknowledgement for a write, its later read should reflect that write or a newer value.
- **MR:** after a client observes version k , its later successful reads should not return a version below k .
- **MW:** writes issued sequentially by one logical client should be applied in that client order.
- **WFR:** a write issued after reading version k should be applied only to a history that contains version k or a later version.

We tested three questions. First, can asynchronous Secondary replication expose stale state under weak settings? Second, do majority concerns and a causally consistent session replace stale success with waiting or explicit unavailability? Third, how do node loss, failover, and network partition change the consistency and availability outcomes? Our experiments provide bounded observations for specified protocols; they do not prove universal guarantees over every possible execution.

2. Database choice and deployment

2.1 Why MongoDB

MongoDB was selected because a replica set provides a practical replicated database while allowing consistency controls to be varied per operation. PyMongo exposes read concern, write concern, read preference, and logical sessions directly. This makes it possible to separate four mechanisms: where a read is routed, what state is visible to it, how durable a write acknowledgement is, and whether causal dependency metadata is carried between operations. MongoDB also supports controlled elections, member shutdown, and asynchronous replication, which are useful for the required failure scenarios.

2.2 Architecture

The deployment contains three voting, data-bearing MongoDB members in replica set rs0. Under healthy operation there is exactly one Primary and two Secondaries. The normal cluster advertises mongo1:27017, mongo2:27017, and mongo3:27017 inside its Docker network, with host mappings on 127.0.0.1 ports 27017-27019. Clients run in the Docker network so replica-set discovery uses the advertised container addresses.

Fault experiments use an independent Compose project and separate volumes. The database members share a replication network, while per-node observation networks retain controlled diagnostic access. The host controller can stop a container or disconnect replication links; the experiment runner cannot access the Docker socket. A lock prevents concurrent fault experiments, and recovery logic in a finally block reconnects networks and restarts nodes. Member roles are discovered dynamically, so no script assumes that mongo1 is always Primary.

Component	Recorded environment
Database	MongoDB 8.0.12, three-member replica set rs0
Main host	Apple M4, 16 GiB RAM, macOS 15.7.3
Container platform	Docker Desktop 4.45.0, Engine 28.3.3, Compose 2.39.2-desktop.1
Docker VM allocation	10 CPU, approximately 7.65 GiB RAM
Experiment runtime	Python 3.12.14, PyMongo 4.17.0
Supporting packages	dnspython 2.8.0, jsonschema 4.26.0
Analysis environment	Python 3.12; Matplotlib 3.11.1 for figures

Some trials were reproduced on another Apple Silicon host with a newer Docker release. Each archived manifest records the actual host-independent software state and source hash; therefore the table describes the main environment rather than silently assigning one host version to every run.

2.3 Installation, startup, and initialization

The reproducible setup requires Git, Docker Compose v2, and Python 3.9 or later for the host controller. The experiment runner and PyMongo dependencies are built into a pinned image:

```
docker build -t dsa5208-experiment-runner:local -f docker/runner.Dockerfile .
./scripts/start-cluster.sh
./scripts/cluster-status.sh
```

The startup script creates the containers, runs rs.initiate when required, and waits for one Primary and two Secondaries. Waiting only for ping is insufficient because a replica set may accept connections before electing its first Primary. A clean-deployment check confirmed initialization, C1/C4 test writes and reads, shutdown, and restart on independent volumes. Before every fault batch the controller verifies cluster topology; after every trial it requires restored membership and document convergence.

3. Consistency configurations and predictions

3.1 Tested configurations

ID	readConcern	writeConcern	readPreference	Explicit causal session	Purpose
C1	local	w=1	secondaryPreferred	No	Weak baseline; exposes replication lag and permits rollback of acknowledged writes
C2	local	w=1	primary	No	Isolates the benefit of reading from the current Primary during stable operation
C3	majority	w=majority	secondaryPreferred	No	Improves durability and visibility without explicitly propagating client dependencies
C4	majority	w=majority	secondaryPreferred	Yes	Strong causal configuration; all dependent operations share one session

`local` allows a member to return its locally available state, which may later be rolled back. A `majority` read returns majority-committed data, although a Secondary may still lag the latest committed point. `w=1` acknowledges after one member accepts the write; `w=majority` waits for majority acknowledgement. `secondaryPreferred` normally selects a Secondary but can fall back to the Primary. For C4, every causally related operation explicitly receives the same `ClientSession` with causal consistency enabled and the session is used by one thread at a time.

C3 and C4 deliberately hold read concern, write concern, and read preference constant. Their intended difference is explicit propagation of causal dependency metadata. An implicit driver session in C1-C3 is not treated as proof that application operations shared a causal chain.

3.2 Predictions registered before formal sampling

Configuration	RYW	MR	MW	WFR
C1	Not guaranteed	Not guaranteed	Expected for ordinary sequential modifying writes	Not guaranteed
C2	Conditional during stable single-Primary execution	Conditional	Expected for ordinary sequential modifying writes	Conditional
C3	Not guaranteed solely from majority concerns	Not guaranteed solely from majority concerns	Expected and majority durable	Not guaranteed solely from majority concerns
C4	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold	Guaranteed when prerequisites hold

For C1, stale Secondary reads were expected to make RYW and MR violations observable when lag was deliberately exposed. MW violations were not expected for sequential writes that actually modify data, but a `w=1` acknowledgement could be rolled back after failover; rollback was therefore classified separately. C2 was expected to look strong under normal operation because both reads and writes target the current Primary, without providing a durable cross-failover causal claim. C3 was expected to improve durability while still lacking an explicit client dependency chain. C4 was expected to wait, fail, or time out rather than successfully return causally older state.

The C4 prediction requires acknowledged predecessor writes, majority read and write concerns, one causally consistent session, serial session use, and explicit passage of that session to every related operation. Tests compare `version`, `client_seq`, and `depends_on` fields rather than wall-clock timestamps. A timeout or write-concern failure is an availability outcome and cannot be counted as either a successful consistency check or a violation.

4. Experimental method

4.1 Common trial record and scenarios

Every trial uses a unique experiment and document identity. Operations record the model, configuration, client and session identifiers, operation type, actual target node, written and returned versions, monotonic duration, status, error, violation decision, and current Primary. Manifests retain source hashes, source archives, runtime versions, effective arguments, and cluster snapshots. Pilot, formal, legacy, and diagnostic cohorts are never pooled.

The scenario labels used in this report are:

Scenario	Fault action and purpose
S1 normal	Healthy one-Primary/two-Secondary replica set; establishes baseline behavior
S2 Secondary stop	Stops one selected Secondary during the causal sequence; exposes routing and lag behavior
S3 two-Secondary stop	Removes the majority; contrasts immediate operation with operation after the Primary steps down
S4 Primary crash	Stops the actual Primary between relevant operations and waits for election where the protocol requires it
S5 replication partition	Isolates replication links while preserving controlled observation paths; exposes divergent histories and rollback

4.2 RYW experiment

Each trial initializes `x.version=0`, then one logical client writes `x.version=1` and waits for acknowledgement. The same client immediately reads `x`. A successful read returning a version below the acknowledged version is a RYW violation. If the write is unacknowledged, times out, or fails, the trial has no valid RYW predecessor and is reported as unavailable or inconclusive.

The weak path uses `w=1`, local reads, `secondaryPreferred` and no explicit causal session. The strong path uses majority acknowledgement, majority reads, and one causal session. S2 is especially useful because the read can reach a Secondary whose visible state differs from the acknowledged write. S3 tests behavior after loss of a majority, where waiting or timeout is expected to dominate.

4.3 MR experiment

Each trial first places `x.version=1` on all required members. An independent producer writes version 2. One logical read client performs its first read, the controller injects S2 or S4, and the same client performs its second read from the eligible topology. A second successful read below the first observed version, or disappearance of a previously observed document, is a violation. C4 uses the same explicit session for both reads.

The fault is deliberately placed between the two reads. In S2, the runner confirms that the stopped member was the Secondary serving the first read when applicable. In S4 it stops the actual Primary and waits for a replacement. This protocol tests monotonicity of one client's observations rather than freshness of the first read: a transition 1 to 2 is monotonic, while 2 to 1 is not.

4.4 MW experiment

One logical client issues write 1 with `version=1` and `client_seq=1`, waits for its outcome, then issues write 2 with `version=2` and `client_seq=2`. The main protocol uses synchronous sequential calls. Atomic `findOneAndUpdate` preimages, write histories, and an independent final audit establish whether the predecessor existed before the second update. Concurrent completion order is not treated as an MW violation.

An acknowledged write that is later rolled back is reported as rollback, not client write-order inversion. A timeout is also separate because the client cannot infer that the write did not take effect. S2 and S3 test whether sequential operations remain available with one or two Secondary losses; historical S4 and S5 diagnostics examine failover and rollback behavior.

4.5 WFR experiment

The fixed continuity protocol first commits `x.version=1` and verifies seed convergence. The tested client reads `x` and records `k`. The controller then injects S2, S4, or S5. The same logical client writes `y.depends_on_x=k`; C4 retains the same session across the read and dependent write. The write records an atomic preimage, and a separate observer checks the marker and the corresponding `x` history.

If a node has observed `y` depending on `x=k` while its `x` version is below `k`, the outcome is a candidate WFR violation requiring corroboration. Missing marker evidence, write timeout, or inability to establish the predecessor is inconclusive.

rather than a pass. The fixed protocol studies continuity of a committed dependency. It is intentionally different from an earlier diagnostic that created an uncommitted dependency on an isolated old Primary; the earlier rollback candidate is retained separately and is not promoted to a confirmed violation.

4.6 Metrics and evidence rules

The violation rate is violations divided by evaluable trials. Candidate, invalid, rollback, and inconclusive outcomes are shown separately; a zero evaluable denominator is N/A, not 0%. Operation success is reported both against attempted workload operations and against planned slots in started trials. Successful-operation p95 excludes timeouts and failures. Recovery success requires explicit restoration of one Primary, two Secondaries, and convergence evidence. Host and container wall clocks are not subtracted from each other; ordering uses same-process monotonic timestamps and matched control receipts.

5. Results

5.1 Normal-operation evidence

Model and cohort	C1	C2	C3	C4	Interpretation
RYW legacy, 100/config	93/100 violations	0/100	1/100	0/100	Weak Secondary routing exposed stale reads; older logs lack reliable operation-level latency
MR baseline, 100/config	0/100	0/100	0/100	0/100	Pilot baseline observed no regression without a targeted lag window
MW baseline, 100/config	0/100	0/100	0/100	0/100	Sequential modifying writes preserved client order in the tested runs
WFR continuity smoke	0/1	Not run	Not run	0/1	Two smoke trials only; not promoted to formal normal-operation evidence

These cohorts differ in schema and sampling stage. They support qualitative baseline comparisons but are not pooled into one formal rate. The RYW C1 result is consistent with immediate Secondary reads exposing lag. The single C3 stale observation shows that majority visibility alone does not carry an explicit client dependency. The absence of normal-operation MR/MW violations does not establish universal guarantees, especially where the experiment did not force nodes to disagree.

5.2 Secondary loss and loss of majority

Scenario and model	C1 violation/evaluable	C4 violation/evaluable	Availability and recovery
S2 RYW formal	70/100	0/100	All 200 trials completed workload operations and recovered
S2 MW formal	0/100	0/100	All planned writes succeeded; 200/200 recovered
S2 MR formal	0/100	0/100	400 workload reads succeeded across 200 trials; 200/200 recovered
S2 WFR formal	0/100	0/100	400/400 workload operations succeeded; 200/200 recovered
S3 immediate MW formal	0/99	N/A, 0/100 evaluable	C1 had one timeout; C4 had 100 first-write timeouts; every trial recovered
S3 immediate RYW formal	0/100	N/A, 0/100 evaluable	C4 produced 99 timeouts and one failure; every trial recovered
S3 settled MW/RYW formal	N/A	N/A	Each model/configuration group had 100 inconclusive attempts after loss of majority

The S2 RYW contrast is the clearest treatment observation: C1 produced stale successful reads while C4 produced none. In S3, zero evaluable C4 trials cannot be described as a zero-percent violation rate. Majority write acknowledgement became unavailable after two Secondaries stopped. This supports the prediction that the strong

configuration may preserve its consistency contract by refusing to complete rather than returning a stale success.

5.3 Primary crash and partition evidence

The formal MR S4 batches completed 100 C1 and 100 C4 trials. Both had 0/100 regressions, no timeout or error, and 100/100 recovery. Successful read p95, pooling first and second successful reads within each group, was 1.186 ms for C1 and 0.914 ms for C4 in this local container environment. These small values do not measure election time because the second read was issued only after the protocol's topology gate.

The targeted MR S5 diagnostic created a visible version difference. C1 produced a 2-to-1 regression in all five trials. C4 produced no evaluable pair because every second read timed out. The contrast agrees with the prediction: weak settings can return older state, while the causal configuration may wait or become unavailable rather than successfully return it. Five trials are diagnostic evidence, not an estimate of a general violation probability.

Historical MW S4 batches used 20 attempts per configuration and observed no confirmed write-order violation. S5 diagnostics separated rollback from ordering: the C1 cohort contained rollback outcomes, while the C4 cohort lacked an evaluable consistency denominator. RYW S4/S5 evidence contains only one pilot attempt per configuration, so it is used as a trace-level diagnostic and not as a central quantitative claim.

5.4 Fixed WFR fault matrix

Formal scenario	C1 violation/evaluable	C4 violation/evaluable	Workload operation success	Recovery
S2 Secondary stop	0/100	0/100	400/400	200/200
S4 Primary crash	0/100	0/98	398/400; two C4 dependent-write timeouts	200/200
S5 replication partition	0/98	0/100	400/400; two C1 attempts lacked audit corroboration	200/200

Across 600 fixed formal attempts, 596 were evaluable and no confirmed or candidate WFR violation was observed. Four attempts were inconclusive, 1,198 of 1,200 workload operations succeeded, and 600 of 600 trials recovered. The separate 120-attempt pilot produced 117 evaluable trials, no violation, three inconclusive outcomes, 239 of 240 successful workload operations, and complete recovery. No failed or inconclusive attempt was replaced.

Successful dependent-write p95 was 2.47/4.60 ms for S2 C1/C4, 10,193.41/10,198.75 ms for S4, and 2.44/1,958.36 ms for S5. S4 includes driver waiting across election. In S5, the protocol waits for the majority-side replacement Primary before the measured write, so its latency boundary differs. These values characterize protocol behavior and must not be read as a clean performance benchmark between concern settings.

5.5 Overall evidence volume

The final unified analysis catalogs 72 source cohorts and normalizes 4,130 trials into 111 groups; 88 groups satisfy the selection rules for the core figures. This count spans legacy, pilot, formal, and diagnostic evidence and is not a single statistical denominator. The largest completed formal subsets are 1,200 Secondary-related MW/Ryw attempts, 400 MR S2/S4 attempts, and 600 WFR S2/S4/S5 attempts. Every headline rate in this report therefore retains its model, configuration, scenario, protocol, and sampling stage.

6. Comparison with predictions

Prediction	Observation	Assessment
C1 can expose stale Secondary reads	RYW S2 returned stale data in 70/100; MR S5 regressed in 5/5	Supported for the controlled workloads
C4 should not return a successful causally older value when prerequisites hold	No C4 RYW S2 stale read; MR S5 second reads timed out; no C4 WFR violation in 298 evaluable formal trials	Consistent with prediction; finite evidence is not proof
Majority loss should reduce availability	S3 C4 MW/RYW produced no evaluable formal trials after two Secondaries stopped	Supported; N/A is an availability result, not a consistency pass
Sequential modifying writes should remain monotonic	No confirmed MW order violation in the evaluated normal, S2, or historical failover cohorts	Supported within the tested protocols
Majority durability and causal ordering are distinct	C3 produced one stale RYW observation in its legacy normal cohort; C3 and C4 differ only by explicit causal-session use	Consistent, although legacy schema and low anomaly frequency limit causal attribution
Recovery does not imply consistency	Rollback and timeout outcomes occurred even though fault trials restored cluster health	Supported; recovery is reported separately

The results show that configuration cannot be interpreted independently of the operation protocol. A C1 trial can appear strong when all nodes are synchronized, while a targeted lag or partition reveals a permitted regression. Conversely, a C4 trial may provide no evaluable result when its required majority or causal dependency is unavailable. Treating every timeout as a consistency success would therefore reverse the meaning of the experiment.

7. Limitations and threats to validity

- **Single-host failure domain.** All three database containers share one physical host, kernel, storage subsystem, and Docker VM. Container or network faults approximate distributed failures but do not reproduce independent machine clocks, disks, and switches.
- **Finite and uneven sampling.** Formal sampling is substantial for selected S2/S3/S4/S5 cells, but several historical S4/S5 cohorts have only one, five, or twenty attempts. No power analysis supports population-level probability claims.
- **Protocol-specific fault windows.** MR injects a fault between reads, while WFR v2 first commits and converges the dependency. Different timing changes the histories that can be observed; results cannot be generalized to every partition schedule.
- **WFR committed-history scope.** The fixed WFR protocol deliberately lowers the chance of missing-history observations by converging x before the fault. It tests continuity of committed history and does not establish a general C1 causal guarantee.
- **Legacy data quality.** Some early RYW logs do not support operation-level latency reconstruction. They remain useful for version outcomes but are excluded from latency claims.
- **Routing controls.** Direct or tagged Secondary access was used in diagnostics to make version differences observable. Such routing is identified separately from default secondaryPreferred behavior.
- **Successful-latency selection.** p95 excludes timeout and failed operations. We therefore report operation success and timeout counts next to latency instead of claiming that the successful subset represents all attempts.
- **Clock boundaries.** Host-controller and runner wall clocks are not assumed synchronized. Fault ordering relies on monotonic timestamps and matched receipts, which bound program order but not the exact physical failure instant.
- **Sequential configuration order.** Several fixed batches run C1 before C4. Thermal, cache, and background-load effects were not randomized, so latency differences are descriptive.

8. Reproduction package

The submission contains the Docker Compose definitions, runner image, Python dependencies, experiment programs, fault-control scripts, analysis code, and selected evidence. A reproducer should first build the runner, start the normal cluster, and verify one Primary and two Secondaries. Every run must use a new experiment ID. Fault experiments must execute serially against the isolated fault project.

Typical verification flow:

```

docker build -t dsa5208-experiment-runner:local -f docker/runner.Dockerfile .
./scripts/start-cluster.sh
./scripts/cluster-status.sh
python -m unittest discover -s tests
python scripts/run-fault-experiment.py --help
python -m analysis.unified --output results/summary/reproduced
python -m analysis.plot_results --input results/summary/reproduced --output results/figures/reproduced

```

The 22 September source state passed 117 unit tests, including oracle edge cases, dispatch compatibility, archived-run validation, and tamper rejection. Unit tests verify software behavior and are not counted as database observations. For actual fault execution, recovery failure stops the batch; interrupted output is retained rather than deleted or replaced.

9. Team contributions and AI use

Zhao Yikun led deployment, replica-set and fault-lab setup, common fault orchestration, MW experiments, Secondary-loss experiments, evidence validation, and deployment documentation. Zhou Jiahao led RYW and MR design and execution, completed MR S2/S4 formal sampling, documented model-specific interpretation, and independently reproduced a small S2 MW workflow. Chen Zheke led WFR protocol design and execution, conservative WFR classification, unified analysis and figures, and report integration. All members are responsible for reviewing the final claims and submitted artefacts.

An OpenAI coding assistant supported code review, implementation suggestions, synthetic tests, command execution, evidence checking, figure generation, and drafting. Team members selected the research questions, operated the database experiments, reviewed the observations, and remain responsible for the final interpretation. AI-generated prose and synthetic fixtures were not treated as empirical evidence; quantitative claims in this report trace to archived database logs and validation outputs.

10. Conclusion

MongoDB's tunable controls produce an observable trade-off between stale success and unavailable operations. Weak local, $w=1$, Secondary-preferring access exposed stale RYW and MR results under controlled lag or partition. Majority concerns plus an explicit causal session produced no successful stale observation in the corresponding tested cohorts, but often waited or timed out when the required history or majority was unavailable. Sequential modifying writes remained monotonic in the evaluated MW trials, while rollback remained a separate durability event. The committed-history WFR matrix observed no violation in 596 evaluable formal trials and four inconclusive attempts.

The central methodological result is that consistency, durability, operation availability, and cluster recovery require separate denominators. A recovered cluster does not retroactively make a timed-out operation successful, and a zero-evaluable group cannot support a zero-percent violation claim. Within those boundaries, the observations agree with the registered predictions and demonstrate how concern settings, routing, sessions, and fault timing jointly shape client-visible consistency.

References

- [1] MongoDB, Inc. "Causal Consistency and Read and Write Concerns." MongoDB Manual. <https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/> (accessed 23 September 2026).
- [2] MongoDB, Inc. "Read Isolation, Consistency, and Recency." MongoDB Manual. <https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/> (accessed 23 September 2026).
- [3] MongoDB, Inc. "Read Concern majority." MongoDB Manual. <https://www.mongodb.com/docs/manual/reference/read-concern-majority/> (accessed 23 September 2026).
- [4] MongoDB, Inc. "Read Preference." MongoDB Manual. <https://www.mongodb.com/docs/manual/core/read-preference/> (accessed 23 September 2026).
- [5] MongoDB, Inc. "Write Concern." MongoDB Manual. <https://www.mongodb.com/docs/manual/reference/write-concern/> (accessed 23 September 2026).
- [6] MongoDB, Inc. "Replication." MongoDB Manual. <https://www.mongodb.com/docs/manual/replication/> (accessed 23 September 2026).
- [7] MongoDB, Inc. "ClientSession - PyMongo API Documentation." https://pymongo.readthedocs.io/en/stable/api/pymongo/client_session.html (accessed 23 September 2026).